ay/date

Quick sort

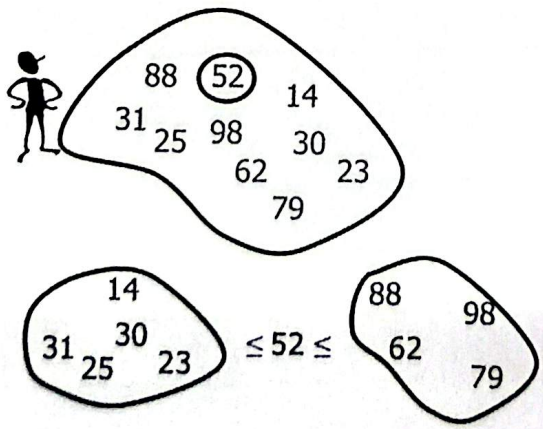
Also uses divide & conquer paradigm.

How it works?

It operates by choosing a pivot element from the array (either randomly or some way) & partitioning the remaining elements into 2.

- left subarray : elements greater than or equal to pivot
- right subarray : elements smaller than pivot.

Partition set into two using randomly chosen pivot



BHATTI

CamScanner

te

Once partitioning is done, the pivot is in its correct final position.

Then recursively all the left & right subarrays are sorted (using same partition mechanism).

BHATTI

CamScanner

ate

Algorithm:

```
QuickSort (A, p, r)
    IF p < r THEN
        q = Partition (A, p, r)   # q is final index of pivot.
        QuickSort (A, p, q-1)     # left subarray
        QuickSort (A, q+1, r)     # right subarray.

Partition (A, p, r)
    x = A[p]   # we are choosing first element as pivot.
    i = p
    FOR j = p+1 TO r
        IF A[j] <= x THEN
            i = i+1
            exchange A[i] with A[j].
    exchange A[p] with A[i]
    # now n = A[p] has been placed in its correct place.
    return i   # pivots final index
```

BHATTI

CamScanner

day/date

Understanding Partition function:

$x = A[p]$ sets the pivot.
$i = p$, points to the start.

The in loop we scan all elements.
If $A[j] \leq x$ i is increments & $A[j]$ is swapped in to the $\leq x$ region.

Then comes final swap $A[p]$ with $A[i]$ which puts the pivot right in middle of 2 regions.

BHATTI

CamScanner

/date

Dry run:

[6, 10, 13, 5, 8, 3, 2, 11]

we will be taking first element as pivot

In start

(A, 1, 8)

p=1 r=8

now coming to partition function:

test condition	Action	value of variable
x=6		i=1
10 < 6 ✗		j=2 , i=1
13 < 6 ✗		j=3 , i=1
5 < 6 ✓	now i++	j=4
	& swap:	
	A[i], A[j]	i=2 now
	6, 5, 13, 10, 8, 3, 2, 11	
8 < 6 ✗		j=7 , i=2
3 < 6 ✓	now i++	j=6
	Swap Ai, Aj	i=3
	6, 5, 3, 10, 8, 13, 2, 11	

BHATTI

CamScanner

y/date

& continue this for all elements at end

6, 5, 3, 2, 8, 13, 10, 11

& then $r = 4$.

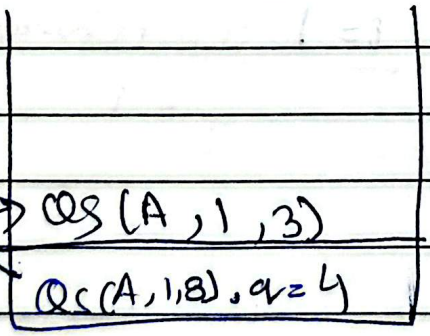
A[p] exchange A[i].

2, 3, 5, 6, 8, 13, 10, 11

Position fixed.

& 4 is returned.

now



& then this returns : calls QS(A, 5, 8).

BHATTI

CamScanner

day/date

Now in QS (A, 1, 3)

A = (2, 5, 3...) → array looks like this
↳ it has other element but we are considering to find 3 only of (A, 1 → 3)

test condition	Action	value of variable
x = 2		j = 2, i = 1
5 ≤ 2 ✗		
3 ≤ 2 ✗		

so no swap

& A[i] = A[p] remain same.

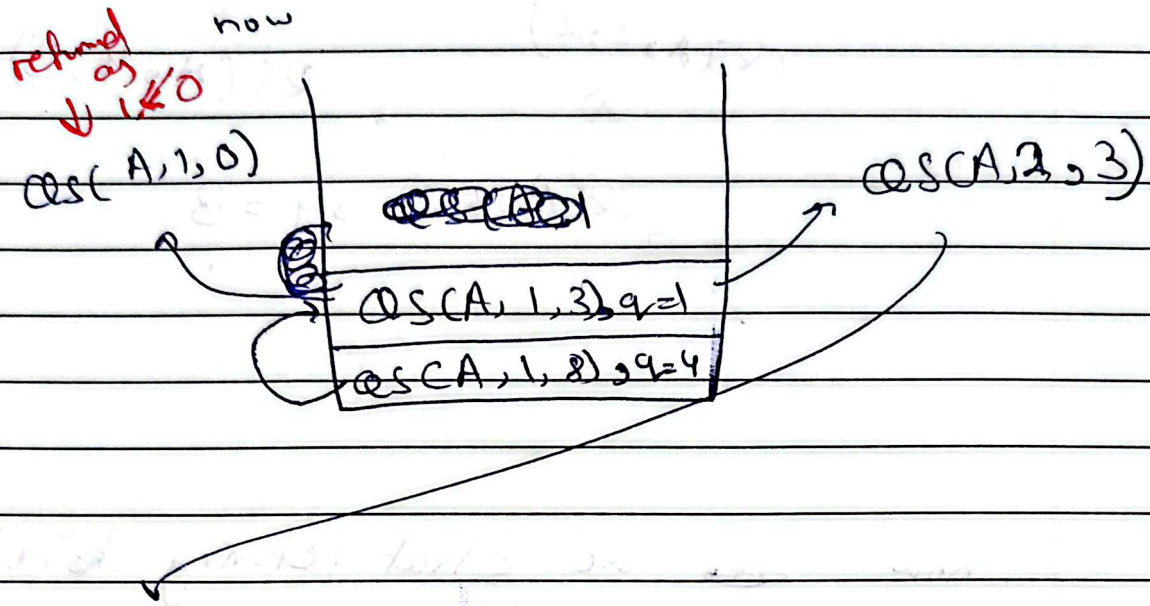
i = 1 ; returned

q = 1

BHATTI

CamScanner

'date



QS (A, 2, 3)

A = (... 5, 3 ...)

now

x = 5

i = p = 2 , r = 3

is

3 < 5 ⟶ yes

so now

(... 3, 5)

& index returned is

3

BHATTI

ay/date

returned & back
base condition pays.

QS(A, 2, 2)

QS(A, 4, 3)

QS(A, 2, 3) , q = 3

how we start returning back to our

QS(A, 1, 8) & q=4 , call.

& array look like

2, 3, 5, 6, 8, 13, 10, 11

right side is sorted.

how comy call:

QS(A, 5, 7)

Continue own self.

bcz you get the idea

BHATTI

CamScanner

ate

Analysis: (space).

As for space complexity it has (log n) due to recursive call on stack other than that it uses no extra arrays.

Analysis: (Time):

worst case:

The worst case occurs when the chosen pivot is the minimum or maximum element in the subarray.

Interestingly, if we are choosing Pivot as first element & array already sorted, that is a worst case scenario.

1, 2, 5, 7, 8
↑ pivot

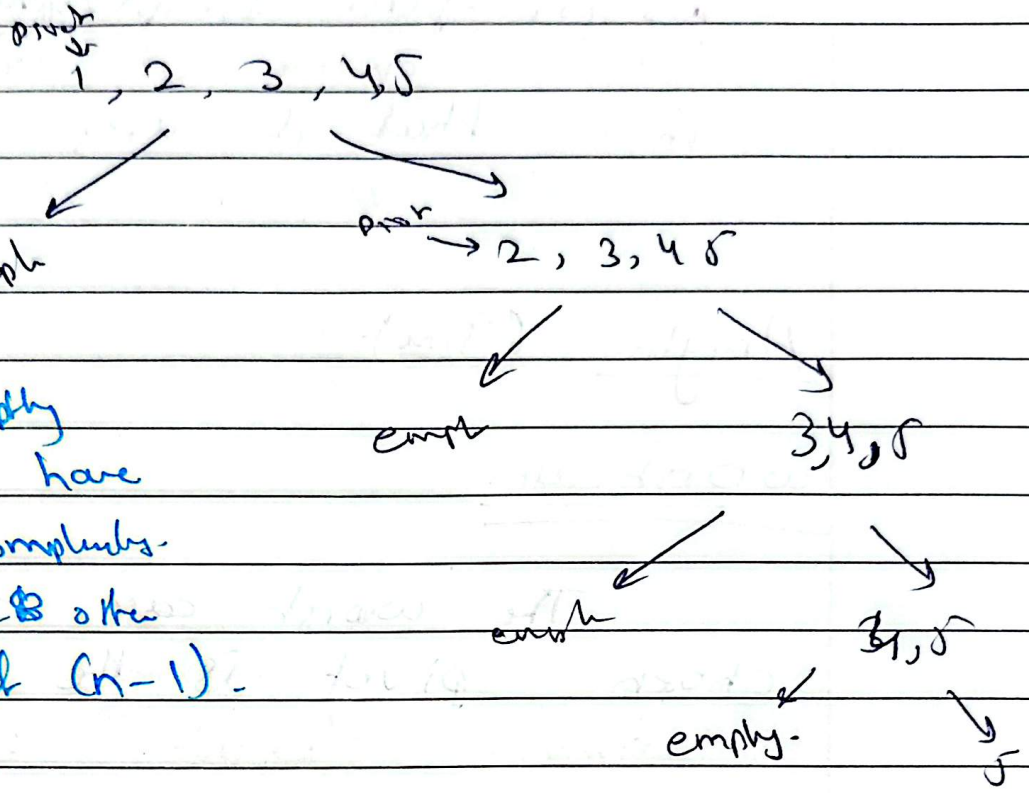
Partition function has complexity of $\Theta(n)$.

BHATTI

CamScanner

day/date

in worst case:



now the empty subarray have T(0) complexity. while the other subarray get (n-1).

so worst case becomes:

$$T(n) = T(n-1) + T(0) + n$$

- $T(n-1)$: subarray is just one less
- $T(0)$: empty subarray.
- n: Partition function

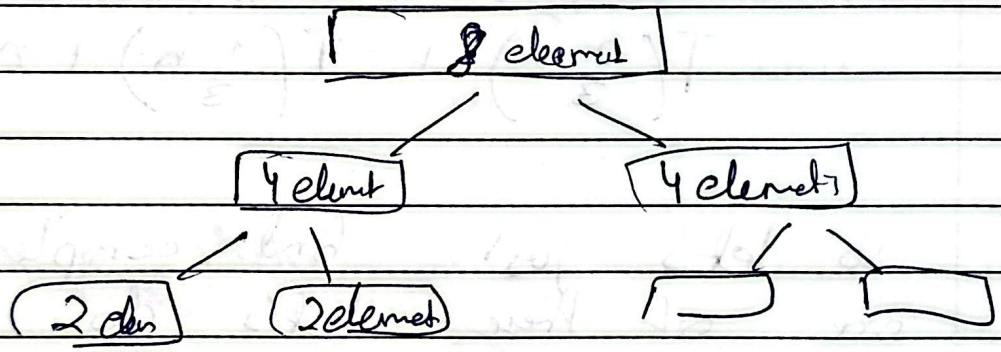
BHATTI

If we solve it we get:

Time complexity in worst case = $\Theta(n^2)$.

Best case:

This happens when our array is divided exactly into 2 equal parts



In this each subarray gets $T(\frac{n}{2})$

So $T(n) = 2T(\frac{n}{2}) + \Theta(n)$.

$2T(\frac{n}{2})$ → both sub arrays

$\Theta(n)$ → partition function.

Time complexity in best case = $\Theta(n \log n)$

BHATTI

day/date

Average case

now what can an average case be?

e.g for 10 elements it can be like

$$T\left(\frac{1}{10}n\right) + T\left(\frac{9}{10}n\right) + n)$$

1/10 one array get one element & other 9

partition function

or

for 3 elements: 2/3 one array get 2 & other 1 : 1

$$T\left(\frac{2}{3}n\right) + T\left(\frac{1}{3}n\right) + n)$$

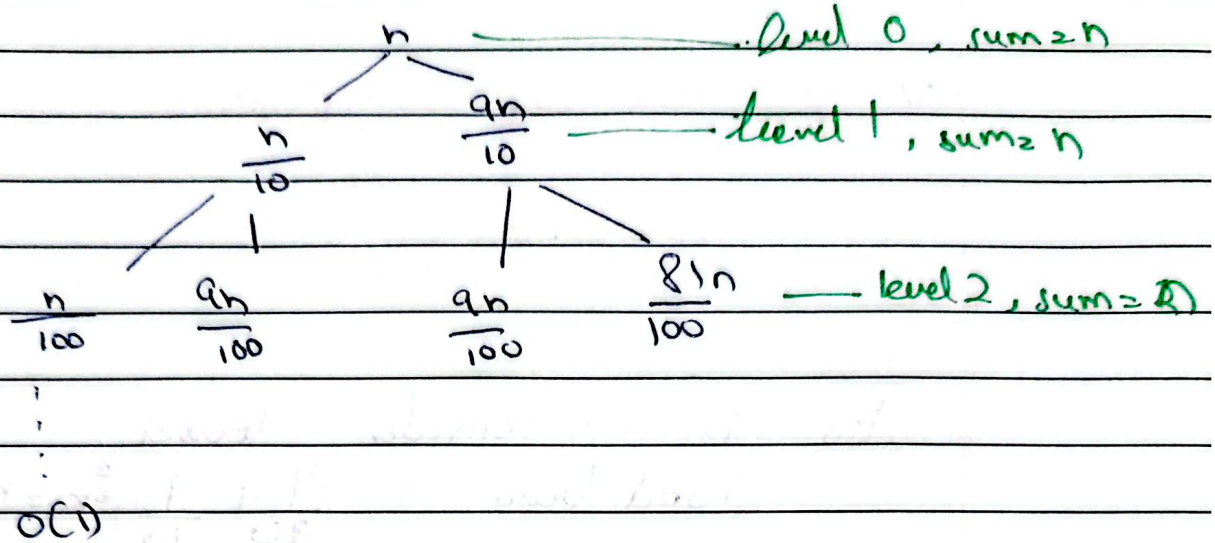
So let's just find complexity of one of these let's do

$$T\left(\frac{1}{10}n\right) + T\left(\frac{9}{10}n\right) + n.$$

BHATTI

CamScanner

ate



As this is an asymmetric tree so we consider biggest branch to find k.

$n \rightarrow \frac{9n}{10} \rightarrow \frac{81}{100} n \cdots O(1)$.

$$\left(\frac{9}{10}\right)^k (n) = 1$$

$$k = \log_{\frac{10}{9}} (n) = k.$$

~~bold a because case case case finding~~

Sum of Internal nodes = kn

$= \log_n (n)$

$= n \log n$

BHATTI

day/date

so avg case complexity:

$\Theta(n \log n)$.

also if we consider leaves:

Total leaves $= \left(\frac{1}{90} + \frac{9}{10}\right)^{\log n} n = n$. → formed by short left.

so leaves cost $= n * 1 = n$.

If we consider shortest branch as well:

$$\Theta(n \log n) \leq x \leq \Theta(n \log n) + \Theta(n)$$

shortest branch

longest branch.

But all shows it is ~~inside~~ $\Theta(n \log n)$.

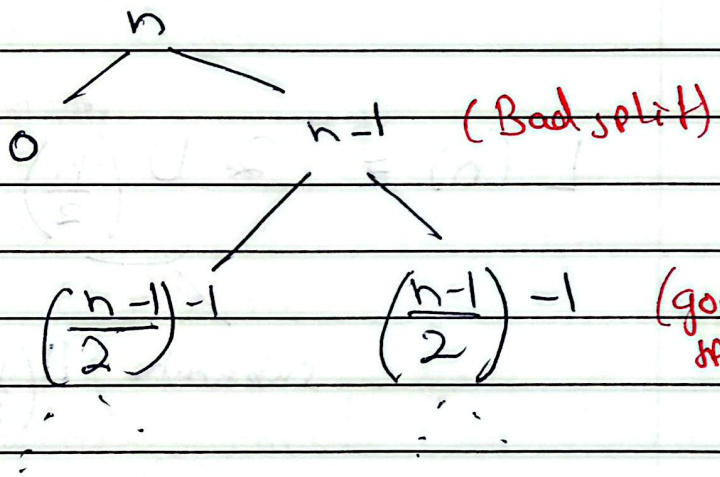
BHATTI

CamScanner

ate

Doing Average case by Alternating good & Bad splits:

consider we pass an array in to quick sort but it inside alternate into good (n/2, branches) & bad splits (0, n-1).



$T(n) = T(n-1) + T(0) + n$ (Bad split).

$T(n) = 2T(n/2) + n$ (good split).

now in alternations:

$Lucky(n) = L(n) = 2U\left(\frac{n}{2}\right) + n$

$Unlucky(n) = U(n) = L(n-1) + n$

BHATTI

day/date

After lucky split, unlucky split come & vice versa due to alternation.

But now how can we solve this?

This is indirect recursion happening.
so use substitution.

$$L(n) = 2U\left(\frac{n}{2}\right) + n$$

substitute $U\left(\frac{n}{2}\right) = L\left(\frac{n}{2} - 1\right) + \frac{n}{2}$

$$L(n) = 2L\left(\frac{n}{2} - 1\right) + 2n$$

after solving this
we will again
get $\Theta(n \log n)$

again average case shows
$\Theta n \log n$.

BHATTI

CamScanner

lay/date

Cases in QuickSort:

We can tweak our Quick Sort algo by changing our pivot selection. & all of this can lead to change in complexities as well.

1) First element as Pivot:

in Partition function:

$x = A[p]$.

we were currently doing this.

→ pivot always median of subarray.

Best case: $\Theta(n \log n)$

Worst case: $\Theta(n^2)$

→ Already sorted or reverse sorted Array.

2) Last element as Pivot:

In partition function

$x = A[r]$

rest Exactly same as Choosing first element. worst & Best cases are also same.

BHATTI

CamScanner

ate

3) Random element as Pivot:

In partition function:

$x = rand() \% (r - p) + p$

Worst case: Still $\Theta(n^2)$ but depends on random selection not input order.

Average Case: $\Theta(n \log n)$ for any input array even sorted ones.

4) Median of 3 pivot selection:

This is the most used way.

In this x is decided based on median of $A[p]$, $A[r]$ & $A[\frac{p+r}{2}]$.

e.g Array $A = [1, 2, 9, 4, 5]$

$A(p)$ $A(\frac{p+r}{2})$ $A[r]$.

so a $\{1, 9, 5\}$.

median = 5 so our

$x = A[r]$

BHATTI

CamScanner

te

Achieved across a much broader range of input because extreme points are mostly rejected.

Best case: $\Theta(n \log n)$

Worst case: $\Theta(n^2)$ but very hard to actually come across such sequence that leads to worst case.

Pros & cons of Quick Sort:

Pros:

- In memory sort as $\Theta(\log n)$ space complexity due to stack
- Easily optimized with heuristics e.g (Median of 3).

Cons:

- Worst case is $\Theta(n^2)$
- Un stable sorting algo.

BHATTI

CamScanner